

Chapter 1

Revenue Documentation

1.1 Introduction

The `revenue.py` script calculates the rate-based revenue requirement from a utility perspective using **Option A: real stream and real WACC**. The nominal rate base (AFUDC capital at COD) is deflated to base year; annual revenue is a constant real (base-year) \$/year; present value uses real WACC. **Rate base (nominal at COD)** = `build_cost_afudc` + `row_cost_afudc` + `env_mitigation_afudc`. The result is written to the batch summary (CSV or JSON) and used in BCR and reporting. Revenue calculation can be disabled via `financial.revenue.rate_based.enabled` in the financing config.

1.2 Method Overview

1.2.1 Purpose

1. Load rate-based revenue parameters from financing data: `enabled` and `allowed_return_rate` (from `financial.revenue.rate_based`); `allowed_return_rate` is interpreted as the *real* return.
2. If not enabled: write zero revenue to the output manager and return.
3. Load project technical details (`delay_years`, `construction_years`, `project_lifetime`) and financing details (WACC real, `inflation_rate`, base year).
4. Get rate base *nominal at COD* (AFUDC capital) from `batch_summary.csv` (CSV: `build_cost_afudc`, `row_cost_afudc`, `env_mitigation_afudc`) or from per-module JSON `total_afudc` (JSON mode).
5. Option A: Deflate rate base to base year: $\text{rate_base_real} = \text{rate_base} / (1 + \text{inflation_rate})^{t_{\text{COD}}}$; compute annual revenue (real) = `rate_base_real` × `allowed_return_rate`; undiscounted total (real) = `annual_revenue_real` × `project_lifetime`.
6. Compute present value of the *real* annual revenue stream using `calculate_present_value`(`annual_revenue_real`, real WACC, `project_lifetime`, `start_year` = COD).

7. Write `revenue_nominal` (undiscounted total in real terms), `revenue_pv`, `annual_revenue` (real), `rate_base`, `rate_base_real`, `rate_base_pv`, `annual_revenue_real`, `allowed_return_rate` to the output manager and call `write_batch_summary()`.

1.2.2 Calculation Approach (Option A: Real Stream, Real WACC)

Rate base *nominal at COD* is **AFUDC capital at COD** (build + ROW capital + environmental mitigation). For the utility benefit, CTCC uses Option A: the revenue stream is defined in **real** terms (constant base-year \$/year) and discounted at **real WACC**, consistent with real discounting elsewhere in CTCC.

1. **COD** (years from base): $t_{\text{COD}} = \text{delay_years} + \text{construction_years} + 1$.
2. **Real rate base** (deflate nominal at COD to base year): $\text{RateBase}_{\text{real}} = \text{RateBase}_{\text{nom}} / (1 + \pi)^{t_{\text{COD}}}$, where π is `inflation_rate`.
3. **Real annual revenue** (constant each year in base-year \$): $\text{AnnualRevenue}_{\text{real}} = \text{RateBase}_{\text{real}} \times \text{allowed_return_rate}$ (allowed return is the real return).
4. **Undiscounted total** (real): $\text{TotalRevenue}_{\text{real}} = \text{AnnualRevenue}_{\text{real}} \times \text{project_lifetime}$.
5. **Present value** of revenue (benefit to utility): PV of the constant real stream from COD over project lifetime, discounted at real WACC.

$$\text{Rate Base (nominal at COD)} = \text{build_cost_afudc} + \text{row_cost_afudc} + \text{env_mitigation_afudc} \quad (1.1)$$

$$t_{\text{COD}} = \text{delay_years} + \text{construction_years} + 1 \quad (1.2)$$

$$\text{RateBase}_{\text{real}} = \frac{\text{RateBase}}{(1 + \pi)^{t_{\text{COD}}}} \quad (1.3)$$

$$\text{AnnualRevenue}_{\text{real}} = \text{RateBase}_{\text{real}} \times \text{allowed_return_rate} \quad (1.4)$$

$$\text{TotalRevenue}_{\text{real}} = \text{AnnualRevenue}_{\text{real}} \times \text{project_lifetime} \quad (1.5)$$

$$\text{Revenue (PV)} = \sum_{t=t_{\text{COD}}}^{t_{\text{COD}} + \text{project_lifetime} - 1} \frac{\text{AnnualRevenue}_{\text{real}}}{(1 + r)^t} \quad (1.6)$$

where π is `inflation_rate` and r is the real WACC.

1.2.3 Inputs and Outputs

Inputs:

- Financing data (YAML or JSON): `financial.revenue.rate_based.enabled`, `financial.revenue.r`, `financial.wacc_nominal`, `inflation_rate`, `base_year` for PV.

- Project technical details: `delay_years`, `construction_years`, `project_lifetime`.
- Rate base (AFUDC capital): from `batch_summary.csv` columns `build_cost_afudc`, `row_cost_afudc`, `env_mitigation_afudc` (CSV mode) or from per-module JSON `total_afudc` (JSON mode); requires `CTCC_SCENARIO_ID` in JSON mode.

Outputs:

- `revenue_nominal` (undiscounted total in real terms), `revenue_pv`, `annual_revenue` (real), `rate_base`, `rate_base_real`, `rate_base_pv`, `annual_revenue_real`, `allowed_return_rate` passed to `CTCCOutputManager.add_revenue(results)`; then `write_batch_summary()`. Used by BCR calculator (benefits side) and in final JSON/CSV summary.

1.2.4 Integration with Other Scripts

- `ctcc.py` — Runs `revenue.py` after `weighted_miles`, `build_costs`, `row_costs`, `environmental_mitigation` so capital costs are available.
- `smart_loaders.py` — `load_financing_details`, `get_financing_data_raw`, `load_project_technical_details` (centralized).
- `financial_utils.py` — `calculate_present_value`, `calculate_cod_year`.
- `smart_output.py` — `CTCCOutputManager` for `add_revenue` and `write_batch_summary`.
- `path_config.py` — `OUTPUTS_DIR` for `batch_summary.csv` and JSON output paths.
- `bcr_calculator.py` — Reads revenue (e.g. `revenue_pv`, `revenue_nominal`) from batch summary or aggregated JSON for BCR benefits.

1.3 Key Functions

1.3.1 `load_rate_based_revenue_parameters()`

Loads `enabled` and `allowed_return_rate` from `get_financing_data_raw()["financial"]["revenue"]["rate_based"]`.
 Defaults: `enabled=False`, `allowed_return_rate=0.10`.

Returns: tuple: (`enabled: bool`, `allowed_return_rate: float`).

1.3.2 `load_project_technical_details()`

Uses centralized `load_project_technical_details` and returns (`delay_years`, `construction_years`, `project_lifetime`) from the project technical details dataclass.

1.3.3 get_rate_base()

Returns rate base (AFUDC capital at COD) = `build_cost_afudc + row_cost_afudc + env_mitigation_afudc`. In JSON mode (when `CTCC_OUTPUT_MODE=json` and `CTCC_SCENARIO_ID` is set), reads `total_afudc` from `json_output_<scenario_id>_build_costs.json`, `row_costs.json`, `environmental_mitigation.json` in `OUTPUTS_DIR`. In CSV mode, reads from `batch_summary.csv` (row matching `scenario_id`, or last row if `scenario_id` not set). Returns 0 if files missing or scenario not found.

Key logic (Option A: real stream, real WACC):

```
1 rate_base = get_rate_base() # nominal at COD
2 cod_year = calculate_cod_year(delay_years, construction_years)
3 rate_base_real = rate_base / (1 + financing.inflation_rate) **
   cod_year
4 annual_revenue_real = rate_base_real * allowed_return_rate
5 revenue_undiscounted_real = annual_revenue_real * project_lifetime
6 revenue_pv = calculate_present_value(
7     annual_revenue_real,
8     financing.wacc_real,
9     project_lifetime,
10    start_year=cod_year,
11 )
```

1.3.4 main()

Checks `load_rate_based_revenue_parameters()`; if not enabled, writes zeros (including `rate_base_real`, `annual_revenue_real`) via `add_revenue` and `write_batch_summary` and returns. Otherwise loads details, gets `get_rate_base()` (nominal at COD), computes real rate base, real annual revenue, undiscounted real total, and PV at real WACC (Option A), then adds results and writes batch summary.

1.4 Example

With `financial.revenue.rate_based.enabled = true`, `allowed_return_rate = 0.10` (real), `inflation_rate = 0.03`, rate base nominal at COD = \$500M, `project_lifetime = 40`, $t_{\text{COD}} = 5$:

- Rate base (real) = $500 / (1.03)^5 \approx \$430.5\text{M}$ (base-year \$).
- Annual revenue (real) = $430.5 \times 0.10 \approx \43.05M per year (constant real \$/year).
- Undiscounted total (real) = $43.05 \times 40 \approx \1722M .
- Revenue PV = present value of \$43.05M per year (real) for 40 years starting at year 5, discounted at real WACC.

Standalone run (after capital scripts have run):

```
1 cd scripts
2 python revenue.py
```